Controllo della concorrenza in pratica

- La conflict-serializabilità
  - è verificabile più rapidamente della view-serializzabilità (con opportune strutture dati, complessità lineare),
- Però
  - sarebbe davvero efficiente se potessimo conoscere il grafo dall'inizio, ma così non è:
  - uno scheduler deve operare “incrementalmente”, cioè ad ogni richiesta di operazione decidere se eseguirla subito oppure fare qualcos'altro;
  - non è praticabile mantenere il grafo, aggiornarlo e verificarne l'aciclicità ad ogni richiesta di operazione
  - si basa sull'ipotesi di commit-proiezione
- Quindi, anche la conflict-serializzabilità
  - è inutilizzabile in pratica in modo diretto

Controllo della concorrenza in pratica, 2

- In pratica, si utilizzano tecniche che
  - garantiscono la conflict-serializzabilità senza dover costruire il grafo
  - non richiedono l'ipotesi della commit-proiezione

- Le tecniche utilizzate
  - 2PL (two-phase locking)
  - timestamp (implementazione "multiversion")

Lock

- Principio: l'utilizzo delle risorse (i dati) è controllato, per evitare aggiornamenti indesiderati:
  - Scritture e letture sono incompatibili
  - Diverse letture sono compatibili fra loro
  - Ogni operazione deve essere preceduta da una "richiesta di lock" che eviti la contestualità di letture e scritture
  - Ogni lettura è preceduta da *r_lock* (lock condiviso da più transazioni) e seguita da *unlock*
  - Ogni scrittura è preceduta da *w_lock* (lock esclusivo, assegnato ad una sola transazione) e seguita da *unlock*
- Quando una transazione prima legge e poi scrive un oggetto, può:
  - richiedere subito un lock esclusivo
  - chiedere prima un lock condiviso e poi uno esclusivo (*lock upgrade*)

Gestione dei lock

- Il *lock manager* riceve richieste di lock dalle transazioni e le accoglie o rifiuta, sulla base delle definizioni sopra, formalizzate nella cosiddetta tavola dei conflitti

richiesta	stato della risorsa: free	r_locked	w_locked
r_lock	OK / r_locked	OK / r_locked	NO / w_locked
w_lock	OK / w_locked	NO / r_locked	NO / w_locked
unlock	-	OK / dipende	OK / free

- Il lock manager gestisce una tabella dei lock, per ricordare la situazione
- Un contatore tiene conto del numero di "lettori"; la risorsa è rilasciata quando il contatore scende a zero
- Se la risorsa non è concessa, vediamo dopo che cosa succede

I lock da soli non bastano

- La perdita di aggiornamento, con i lock

t_1	t_2
rlock_1 (x)	
$r_1(x)$	
$x = x - 1000$	
unlock_1 (x)	
	rlock_2 (x)
	$r_2(x)$
	$x = x - 1000$
	unlock_2 (x)
wlock_1 (x)	
$w_1(x)$	
unlock_1 (x)	
`commit`	
	wlock_2 (x)
	$w_2(x)$
	unlock_2 (x)
	`commit`

- Non cambia niente, il problema resta

Locking a due fasi (2PL)

- Oltre alla protezione con lock di tutte le letture e scritture
  - vincolo sulle richieste e i rilasci dei lock:
    - **una transazione, dopo aver rilasciato un lock, non può acquisirne altri**
- "Due fasi"
  - prima fase: acquisizione lock
  - seconda fase: rilascio lock
- Garantisce la conflict-serializzabilità (condizione sufficiente):
  - ogni schedule ammesso dal 2PL è conflict-serializzabile, vediamo

2PL e CSR

- **Ogni schedule 2PL è anche conflict serializzabile, ma non necessariamente viceversa**
- Controesempio per la non necessità:

$$r_1(x)\ w_1(x)\ r_2(x)\ w_2(x)\ r_3(y)\ w_1(y)$$

  - Viola 2PL (uno scheduler 2PL non lo può produrre)
  - Conflict-serializzabile

2PL e CSR

- **Ogni schedule 2PL è anche conflict serializzabile, ma non necessariamente viceversa**
- Controesempio per la non necessità:

$$r_1(x)\ w_1(x)\ r_2(x)\ w_2(x)\ r_3(y)\ w_1(y)$$

  - Viola 2PL (uno scheduler 2PL non lo può produrre)
    - perché?
  - Conflict-serializzabile

2PL e CSR

- **Ogni schedule 2PL è anche conflict serializzabile, ma non necessariamente viceversa**
- Controesempio per la non necessità:

$$r_1(x)\ w_1(x)\ r_2(x)\ w_2(x)\ r_3(y)\ w_1(y)$$

  - Viola 2PL (uno scheduler 2PL non lo può produrre)
    - perché? Rilascia lock su x acquisisce lock su y
  - Conflict-serializzabile

2PL e CSR

- **Ogni schedule 2PL è anche conflict serializzabile, ma non necessariamente viceversa**
- Controesempio per la non necessità:

$$r_1(x)\ w_1(x)\ r_2(x)\ w_2(x)\ r_3(y)\ w_1(y)$$

  - Viola 2PL (uno scheduler 2PL non lo può produrre)
  - Conflict-serializzabile
    - perché?

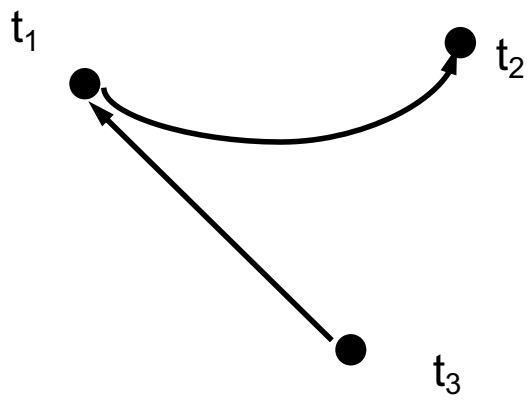
2PL e CSR

- **Ogni schedule 2PL è anche conflict serializzabile, ma non necessariamente viceversa**
- Controesempio per la non necessità:

$$r_1(x)\ w_1(x)\ r_2(x)\ w_2(x)\ r_3(y)\ w_1(y)$$

  – Viola 2PL (uno scheduler 2PL non lo può produrre)

  – Conflict-serializzabile
    - perché?



2PL e CSR

- **Ogni schedule 2PL è anche conflict serializzabile, ma non necessariamente viceversa**
- Controesempio per la non necessità:

$$r_1(x)\ w_1(x)\ r_2(x)\ w_2(x)\ r_3(y)\ w_1(y)$$

- Sufficienza: vediamo

2PL implica CSR

- S schedule 2PL
- Consideriamo per ciascuna transazione l'istante in cui ha tutti i lock e sta per rilasciare il primo
- Consideriamo lo schedule seriale S* in cui ordiniamo le transazioni in accordo il valore temporale di cui sopra
- Dimostriamo che lo schedule seriale S* è equivalente ad S:
  - consideriamo un conflitto in S* fra un'azione di t_i e un'azione di t_j con $i<j$;
  - è possibile che compaiano in ordine invertito in S?
    - no, perché in tal caso t_j dovrebbe aver rilasciato la risorsa in questione prima della sua acquisizione da parte di t_i

Concorrenza e fallimento di transazioni

- Rimuoviamo l'ipotesi di "commit-proiezione"
- Le transazioni possono fallire
- Conseguenze negative: rischio di:
  - letture sporche:
    - se t_i ha letto un dato scritto da t_k e t_k fallisce, ma nel frattempo t_i è andata in commit (o anche solo comunica all'esterno), allora abbiamo l'anomalia

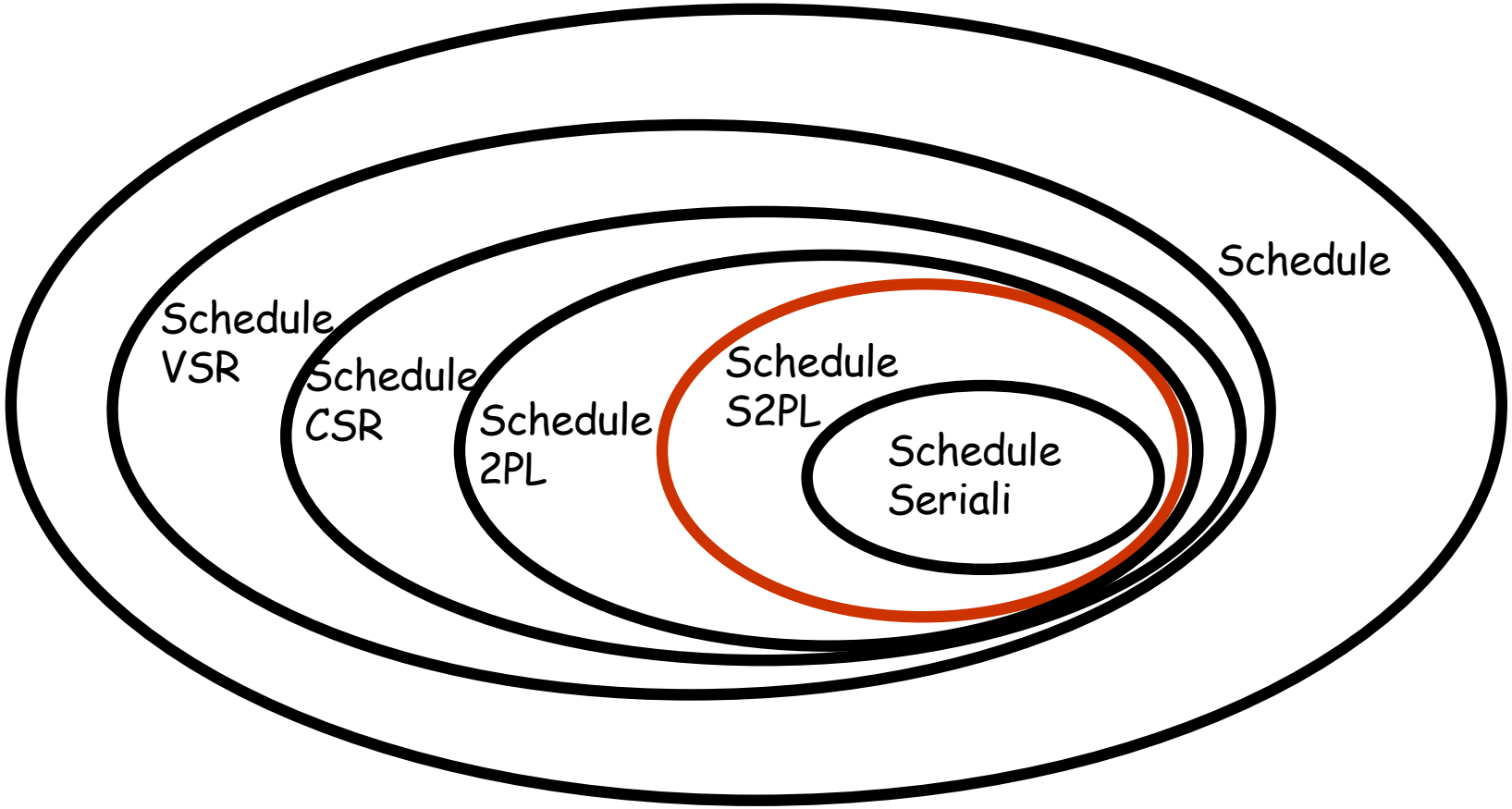
Letture sporche

- Come le evitiamo:
  - una transazione non può andare in commit e non può comunicare all'esterno ciò che ha letto, finché non sono andate in commit tutte le transazioni da cui ha letto
  - meglio ancora: non fare leggere dati sporchi:
    - ogni transazione mantiene i lock fino al commit (o abort)

Locking a due fasi stretto (S2PL)

- 2PL con una condizione aggiuntiva:
  - **i lock possono essere rilasciati solo dopo il commit o l'abort**

CSR, VSR e 2PL


Schedule
Schedule VSR
Schedule CSR
Schedule 2PL
Schedule S2PL
Schedule Seriali

Gestione dei lock

- Ad ogni richiesta di lock è associato un timeout (cioè un tempo massimo di attesa)
- Se una richiesta non può essere soddisfatta (cioè se un lock non può essere concesso)
  - la transazione viene posta in attesa, per un tempo pari al timeout,
  - se la risorsa si libera, la transazione la può acquisire (può, perché ce ne potrebbero essere altre in attesa)
  - se non si libera entro il timeout, la transazione viene abortita; potrebbe essere rilanciata (ma questa è una scelta applicativa)

Granularità e gerarchie di locking

- Nei sistemi reali il lock può essere specificato a diversi livelli di granularità:
  - intera base di dati, tabelle, frammenti, blocchi, ennuple
  - esiste una gerarchia fra le varie granularità
- Perché?
  - Granularità grossa: blocca molte risorse
  - Granularità fine: richiede molti lock

- I sistemi prevedono meccanismi di "escalation" dei lock
  - Ad esempio:
    - se ho molti (troppi) lock su ennuple di una relazione, passo ad un lock sull'intera relazione

Stallo (deadlock)

- Attese incrociate: due transazioni detengono ciascuna una risorsa e aspetttano la risorsa detenuta dall'altra
- Esempio:
  - t_1: $r(x)$, $w(y)$
  - t_2: $r(y)$, $w(x)$
  - Schedule:

    $r_lock_1(x)$, $r_lock_2(y)$, $r_1(x)$, $r_2(y)$, $w_lock_1(y)$, $w_lock_2(x)$

Gestione dello stallo

- Uno stallo corrisponde ad un ciclo nel grafo delle attese (nodo=transazione, arco=attesa)
- Tre tecniche
  1. Timeout (problema: scelta dell'intervallo, con trade-off)
  2. Rilevamento dello stallo
  3. Prevenzione dello stallo
- Rilevamento: ricerca di cicli nel grafo delle attese
- Prevenzione: uccisione di transazioni "sospette" (può esagerare)

2PL nei DBMS

- Abbiamo diversi livelli di isolamento
- Come vengono gestiti?

Livelli di isolamento in SQL:1999 (e JDBC)

- Le transazioni possono essere definite **`read-only`** (non possono scrivere)
- Il livello di isolamento può essere scelto per ogni transazione
  - **`read uncommitted`** permette letture sporche, letture inconsistenti (e aggiornamenti fantasma) e inserimenti fantasma
  - **`read committed`** evita letture sporche ma permette letture inconsistenti (e aggiornamenti fantasma) e inserimenti fantasma
  - **`repeatable read`** evita tutte le anomalie esclusi gli inserimenti fantasma
  - **`serializable`** evita tutte le anomalie
- Nota:
  - la perdita di aggiornamento <u>dovrebbe</u> essere sempre evitata, <u>ma non è così</u>: conviene usare sempre **`serializable`** per le transazioni che scrivono

Anomalie

- `read uncommitted`

Lettura sporca	R-W (o W-W) con abort

- `read committed`

Letture inconsistenti	R-W
Aggiornamento fantasma	R-W

- `repeatable read`

Inserimento fantasma	R-W su dato "nuovo"

- `serializable`

Livelli di isolamento: implementazione

- Sulle scritture si ha sempre il 2PL stretto (e nonostante ciò che dicono i manuali non sempre si evita la perdita di aggiornamento; le transazioni in scrittura debbono avere il livello massimo di isolamento)
- **`read uncommitted`**:
  - nessun lock in lettura (e quindi non rispetta i lock altrui)
- **`read committed`**:
  - lock in lettura (e rispetta i lock in scrittura), **ma senza 2PL**
- **`repeatable read`**:
  - **2PL stretto anche in lettura**, con lock sui dati
- **`serializable`**:
  - i lock visti finora non bastano, perché ci sono record "nuovi"

Inserimento fantasma (phantom)

- Esempio: sistema di prenotazione

t_1	t_2
"conta i posti disponibili sul volo X"	
select … from … where volo=X	
	"aggiungi nuovi posti" (aereo più grande)
	`commit`
"conta i posti disponibili sul volo X"	

Serve impedire l'inserimento di nuove ennuple per il volo X
"**Lock di predicato**" sulla condizione volo=X

Lock di predicato

- Come si implementano?
  - Caso peggiore:
    - sull'intera relazione
  - Se esiste un indice sugli attributi nella condizione:
    - sull'indice

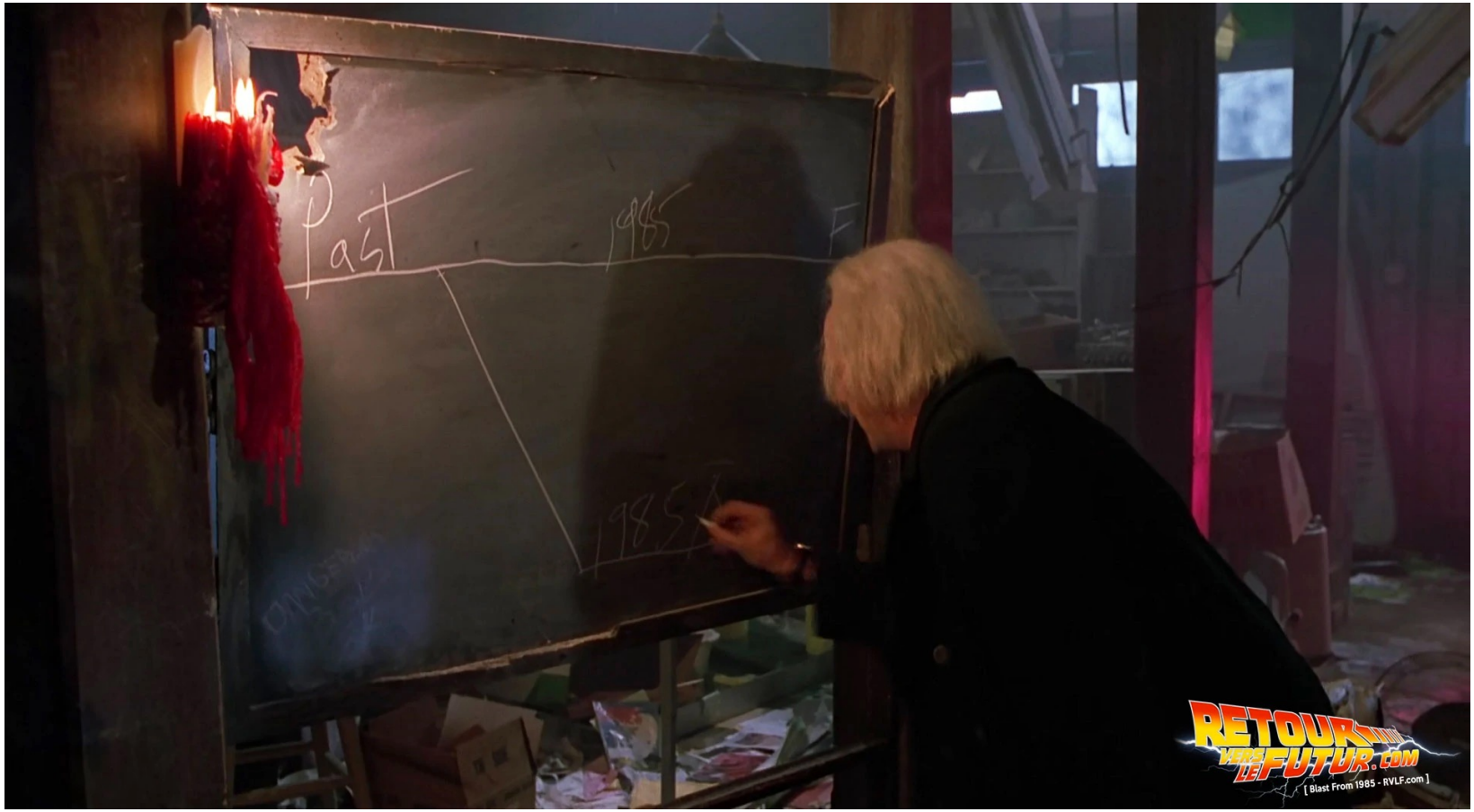
Livelli di isolamento: implementazione

- Sulle scritture si ha sempre il 2PL stretto (e nonostante ciò che dicono i manuali non sempre si evita la perdita di aggiornamento; le transazioni in scrittura debbono avere il livello massimo di isolamento)
- **`read uncommitted`**:
  - nessun lock in lettura (e quindi non rispetta i lock altrui)
- **`read committed`**:
  - lock in lettura (e rispetta i lock in scrittura), **ma senza 2PL**
- **`repeatable read`**:
  - **2PL stretto anche in lettura**, con lock sui dati
- **`serializable`**:
  - 2PL stretto con **lock di predicato**
    - **viene impedito l'accesso in scrittura a tutti i dati che soddisfano il predicato usato nella lettura**

Controllo di concorrenza basato su timestamp

- Tecnica alternativa al 2PL
  - in pratica su usa una sua variante (vediamo più avanti)
- **Timestamp**:
  - identificatore che definisce un ordinamento totale sugli eventi di un sistema
- Intuizione (anche se in pratica la tecnica è un po' diversa)
  - Ogni transazione ha un timestamp che rappresenta l'istante di inizio della transazione
  - Uno schedule è accettato (TS) solo se riflette (nei conflitti) l'ordinamento seriale delle transazioni indotto dai timestamp
    - quindi ogni schedule TS è anche CSR

Timestamp al cinema: Ritorno al futuro



- Vedere anche il "Paradosso del nonno"
  - https://it.wikipedia.org/wiki/Paradosso_del_nonno

Dettagli

- Per ogni oggetto, due variabili RTM(x) e WTM(x)
  - RTM(x): timestamp della transazione più "giovane" che ha letto x
  - WTM(x): timestamp della transazione più "giovane" che ha scritto x

Dettagli

- Per ogni oggetto, due variabili RTM(x) e WTM(x)
  - RTM(x): timestamp della transazione più "giovane" che ha letto x
  - WTM(x): timestamp della transazione più "giovane" che ha scritto x
- Lo scheduler riceve richieste di letture e scritture (t indica il timestamp della transazione che richiede l'operazione):
  - $r_t(x)$:
  - $w_t(x)$:

Dettagli

- Per ogni oggetto, due variabili RTM(x) e WTM(x)
  - RTM(x): timestamp della transazione più "giovane" che ha letto x
  - WTM(x): timestamp della transazione più "giovane" che ha scritto x
- Lo scheduler riceve richieste di letture e scritture (t indica il timestamp della transazione che richiede l'operazione):
  - $r_t(x)$:
    - se t < WTM(x) allora la richiesta è respinta e la transazione viene uccisa;
    - altrimenti, la richiesta viene accolta e RTM(x) è posto uguale al maggiore fra RTM(x) e t
  - $w_t(x)$:

Dettagli

- Per ogni oggetto, due variabili RTM(x) e WTM(x)
  - RTM(x): timestamp della transazione più "giovane" che ha letto x
  - WTM(x): timestamp della transazione più "giovane" che ha scritto x
- Lo scheduler riceve richieste di letture e scritture (t indica il timestamp della transazione che richiede l'operazione):
  - $r_t(x)$:
    - se t < WTM(x) allora la richiesta è respinta e la transazione viene uccisa;
    - altrimenti, la richiesta viene accolta e RTM(x) è posto uguale al maggiore fra RTM(x) e t
  - $w_t(x)$:
    - se t < WTM(x) o t < RTM(x) allora la richiesta è respinta e la transazione viene uccisa,
    - altrimenti, la richiesta viene accolta e WTM(x) è posto uguale a t

Dettagli

- Per ogni oggetto, due variabili RTM(x) e WTM(x)
  - RTM(x): timestamp della transazione più "giovane" che ha letto x
  - WTM(x): timestamp della transazione più "giovane" che ha scritto x
- Lo scheduler riceve richieste di letture e scritture (t timestamp della transazione che richiede l'operazione):
  - $r_t(x)$:
    - se t < WTM(x) allora la richiesta è respinta e la transazione viene uccisa;
    - altrimenti, la richiesta viene accolta e RTM(x) è posto uguale al maggiore fra RTM(x) e t
  - $w_t(x)$:
    - se t < WTM(x) o t < RTM(x) allora la richiesta è respinta e la transazione viene uccisa,
    - altrimenti, la richiesta viene accolta e WTM(x) è posto uguale a t

- Per funzionare anche senza ipotesi di commit-proiezione, deve proteggere le scritture fino al commit (di solito con 2PL sulle scritture)

Piccola variante

- Per le scritture abbiamo scritto
  - $w_t(x)$:
    - se $t <$ WTM(x) o $t <$ RTM(x) allora la richiesta è respinta e la transazione viene uccisa,
    - altrimenti, la richiesta viene accolta e WTM(x) è posto uguale a t
- In effetti, le scritture che arrivano in ritardo, se nessuno ha letto il dato, possono semplicemente essere ignorate; quindi possiamo modificare la regola:
  - $w_t(x)$:
    - se $t <$ RTM(x) allora la richiesta è respinta e la transazione viene uccisa,
    - se $t <$ WTM(x) allora la richiesta è viene ignorata (nessuno ha avuto necessità di leggere quel dato)
    - altrimenti, la richiesta viene accolta e WTM(x) è posto uguale a t

Esempio

$\text{RTM}(x) = 7$

$\text{WTM}(x) = 4$

Richiesta	Risposta	Nuovo valore

Esempio

$\text{RTM}(x) = 7$

$\text{WTM}(x) = 4$

Richiesta	Risposta	Nuovo valore
$r_6(x)$		

Esempio

$\text{RTM}(x) = 7$

$\text{WTM}(x) = 4$

Richiesta	Risposta	Nuovo valore
$r_6(x)$	ok	
$r_8(x)$		

Esempio

$\text{RTM}(x) = \cancel{7}\ 8$

$\text{WTM}(x) = 4$

Richiesta	Risposta	Nuovo valore
$r_6(x)$	ok	
$r_8(x)$	ok	$\text{RTM}(x) = 8$
$r_9(x)$		

Esempio

RTM(x) = ~~7~~ ~~8~~ 9

WTM(x) = 4

Richiesta	Risposta	Nuovo valore
$r_6(x)$	ok	
$r_8(x)$	ok	RTM(x) = 8
$r_9(x)$	ok	RTM(x) = 9
$w_8(x)$		

Esempio

RTM(x) = ~~7~~ ~~8~~ 9

WTM(x) = 4

Richiesta	Risposta	Nuovo valore
$r_6(x)$	ok	
$r_8(x)$	ok	RTM(x) = 8
$r_9(x)$	ok	RTM(x) = 9
$w_8(x)$	no, t_8 uccisa	
$w_{12}(x)$		

Esempio

RTM(x) = ~~7~~ ~~8~~ 9

WTM(x) = ~~4~~ 12

Richiesta	Risposta	Nuovo valore
$r_6(x)$	ok	
$r_8(x)$	ok	RTM(x) = 8
$r_9(x)$	ok	RTM(x) = 9
$w_8(x)$	no, t_8 uccisa	
$w_{12}(x)$	ok	WTM(x) = 12
$r_{10}(x)$		

Esempio

RTM(x) = ~~7~~ ~~8~~ 9

WTM(x) = ~~4~~ 12

Richiesta	Risposta	Nuovo valore
$r_6(x)$	ok	
$r_8(x)$	ok	RTM(x) = 8
$r_9(x)$	ok	RTM(x) = 9
$w_8(x)$	no, t_8 uccisa	
$w_{12}(x)$	ok	WTM(x) = 12
$r_{10}(x)$	no, t_{10} uccisa	
$w_{11}(x)$		

Esempio

RTM(x) = ~~7~~ ~~8~~ 9

WTM(x) = ~~4~~ 12

Richiesta	Risposta	Nuovo valore
$r_6(x)$	ok	
$r_8(x)$	ok	RTM(x) = 8
$r_9(x)$	ok	RTM(x) = 9
$w_8(x)$	no, t_8 uccisa	
$w_{12}(x)$	ok	WTM(x) = 12
$r_{10}(x)$	no, t_{10} uccisa	
$w_{11}(x)$	no, t_{11} uccisa, secondo il metodo base ignorata, secondo la variante	

2PL vs TS

- Sono incomparabili
  - Schedule in TS ma non in 2PL

    $r_1(x)\ w_1(x)\ r_2(x)\ w_2(x)\ r_0(y)\ w_1(y)$

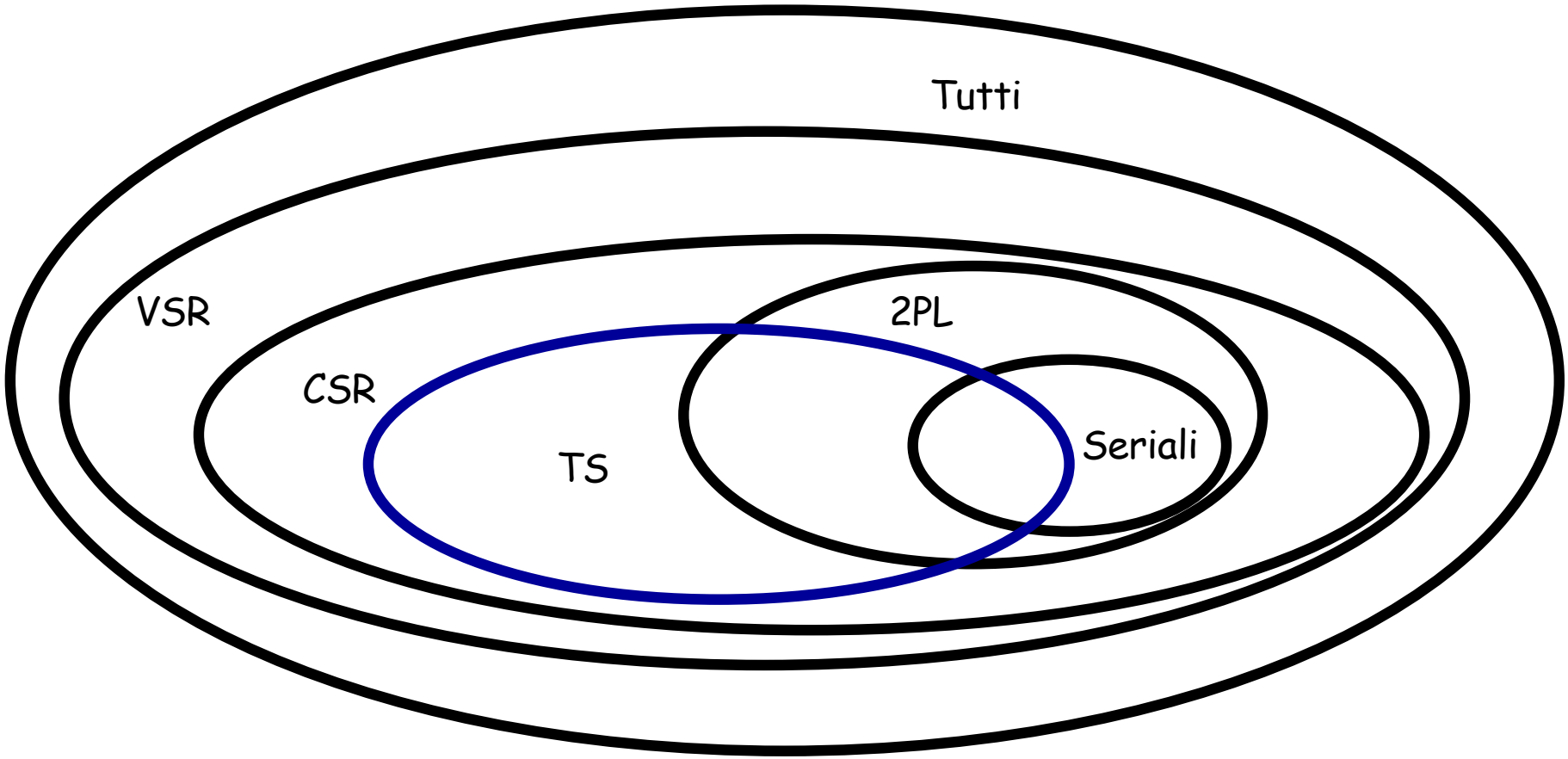
  - Schedule in 2PL ma non in TS (i pedici sono i timestamp)

    $r_2(x)\ w_2(x)\ r_1(x)\ w_1(x)$

  - Schedule in TS e in 2PL

    $r_1(x)\ r_2(y)\ w_2(y)\ w_1(x)\ r_2(x)\ w_2(x)$

CSR, VSR, 2PL e TS



2PL vs TS

- 2PL è "pessimista":
  - In 2PL le transazioni sono poste in attesa
- TS è "ottimista":
  - Va avanti e poi se necessario uccide le transazioni (che, su base delle caratteristiche applicative possono essere rilanciate)
- Per rimuovere la commit proiezione, attesa per il commit in entrambi i casi
- 2PL può causare deadlock (TS anche, ma meno perché non ha i lock in lettura)
- Le ripartenze sono più o meno costose delle attese?
  - originariamente quasi tutti i sistemi preferivano il 2PL
  - ora si stanno diffondendo quelli con timestamp e multiversioni (vediamo fra poco) e, soprattutto, quelli che usano 2PL per le transazioni che scrivono e multiversioni per le read-only

Controllo di concorrenza multiversione, motivazione

RTM(x) = ~~7~~ ~~8~~ 9

WTM(x) = ~~4~~ 12

Richiesta	Risposta	Nuovo valore
$r_6(x)$	ok	
$r_8(x)$	ok	RTM(x) = 8
$r_9(x)$	ok	RTM(x) = 9
$w_8(x)$	no, t_8 uccisa	
$w_{12}(x)$	ok	WTM(x) = 12
$r_{10}(x)$	no, t_{10} uccisa	
$w_{11}(x)$	secondo il met secondo la variante ignorata	

È proprio necessario, uccidere t_{10} ?
Potremmo fare qualcos'altro?

Leggere il valore precedente!

Controllo di concorrenza multiversioni

- Idea base: le scritture generano nuove copie e le letture leggono la copia "valida" per la transazione
  - Una scrittura genera una nuova copia con un WTM.
  - In generale, ci sono più versioni di un oggetto, con timestamp di scrittura diversi
  - Ogni oggetto *x* ha un timestamp di lettura RTM(*x*)

Controllo di concorrenza multiversioni

- Idea base: le scritture generano nuove copie e le letture leggono la copia "valida" per la transazione
  - Una scrittura genera una nuova copia con un WTM.
  - In generale, ci sono più versioni di un oggetto, con timestamp di scrittura diversi
  - Ogni oggetto *x* ha un timestamp di lettura RTM(*x*)
- Meccanismo:
  - $r_t(x)$: è sempre accettata. Viene scelta la copia x_k tale che:
    - se *t* è maggiore di tutti i timestamp delle copie di allora viene scelta l'ultima versione
    - altrimenti, si sceglie la versione k tale che
      - $WTM_k(x) < t < WTM_{k+1}(x)$

Controllo di concorrenza multiversioni

- Idea base: le scritture generano nuove copie e le letture leggono la copia "valida" per la transazione
  - Una scrittura genera una nuova copia con un WTM.
  - In generale, ci sono più versioni di un oggetto, con timestamp di scrittura diversi
  - Ogni oggetto *x* ha un timestamp di lettura RTM(*x*)
- Meccanismo:
  - $r_t(x)$: è sempre accettata. Viene scelta la copia x_k tale che:
    - se *t* è maggiore di tutti i timestamp delle copie di allora viene scelta l'ultima versione
    - altrimenti, si sceglie la versione k tale che
      - $WTM_k(x) < t < WTM_{k+1}(x)$
  - $w_t(x)$: se $t < RTM(x)$ la richiesta è rifiutata, altrimenti una nuova versione di x viene generata, con $WTM(x) = t$

Controllo di concorrenza multiversioni

- Idea base: le scritture generano nuove copie e le letture leggono la copia "valida" per la transazione
  - Una scrittura genera una nuova copia con un WTM.
  - In generale, ci sono più versioni di un oggetto, con timestamp di scrittura diversi
  - Ogni oggetto *x* ha un timestamp di lettura RTM(*x*)
- Meccanismo:
  - $r_t(x)$: è sempre accettata. Viene scelta la copia x_k tale che:
    - se *t* è maggiore di tutti i timestamp delle copie di allora viene scelta l'ultima versione
    - altrimenti, si sceglie la versione k tale che
      - $WTM_k(x) < t < WTM_{k+1}(x)$
  - $w_t(x)$: se $t < RTM(x)$ la richiesta è rifiutata, altrimenti una nuova versione di x viene generata, con $WTM(x) = t$
- Versioni precedenti vengono eliminate quando nessuno è più interessato a leggerle

Controllo di concorrenza multiversioni

- Idea base: le scritture generano nuove copie e le letture leggono la copia "valida" per la transazione
  - Una scrittura genera una nuova copia con un WTM.
  - In generale, ci sono più versioni di un oggetto, con timestamp di scrittura diversi
  - Ogni oggetto *x* ha un timestamp di lettura RTM(*x*)
- Meccanismo:
  - $r_t(x)$: è sempre accettata. Viene scelta la copia x_k tale che:
    - se *t* è maggiore di tutti i timestamp delle copie di allora viene scelta l'ultima versione
    - altrimenti, si sceglie la versione k tale che
      - $WTM_k(x) < t < WTM_{k+1}(x)$
  - $w_t(x)$: se $t < RTM(x)$ la richiesta è rifiutata, altrimenti una nuova versione di x viene generata, con $WTM(x) = t$
- Versioni precedenti vengono eliminate quando nessuno è più interessato a leggerle
- **Attenzione: non è proprio questa la tecnica che si utilizza in pratica! Vedi oltre per Postgres**

Controllo di concorrenza in DBMS reali

- DB2
  - sostanzialmente 2PL stretto, anche se ci sono dettagli che tralasciamo; comunque abbiamo già visto
- PostgesSQL:
  - multiversione, con la condizione "a serializable transaction cannot modify or lock rows changed by other transactions after the serializable transaction began" (tecnicamente, questa è la regola per "repeatable read", vedi più avanti)
- In entrambi, diversi livelli di isolamento anche per le transazioni che scrivono (cosa che non si dovrebbe fare)

Controllo di concorrenza in Postgres: multiversion

- **Read Committed** (Read Uncommitted non è implementato, si comporta nello stesso modo di Read Committed), default:
  - Letture su dati in commit al momento della lettura
    - SELECT vede solo dati andati **in commit prima dell'inizio della SELECT stessa**, a parte gli aggiornamenti della transazione in cui è inserita
  - Scritture con 2PL stretto (richiede lock, e li tiene fino al commit; e rispetta i lock altrui)

Controllo di concorrenza in Postgres: multiversion

- **Repeatable Read**
  - Letture multiversion
    - su dati in commit **all'inizio della transazione**, a parte gli aggiornamenti della transazione in cui è inserita
  - Scritture con 2PL stretto e rispetto delle versioni (una transazione T non può avere lock o modificare dati che siano stati modificati da un'altra transazione T' dopo l'avvio di T)

- Nota bene:
  - Il sistema considera come inizio della transazione non la BEGIN TRANSACTION ma la prima lettura o scrittura della transazione

Controllo di concorrenza in Postgres: multiversion

- **Serializable**
  - Letture multiversion
    - su dati in commit all'inizio della transazione, a parte gli aggiornamenti della transazione in cui è inserita
  - Scritture con 2PL stretto e rispetto delle versioni
    - una transazione T non può modificare dati che siano stati modificati da un'altra transazione T' dopo l'avvio di T) e
    - verifica di cicli di conflitti lettura/scrittura ("read/write dependencies among transactions")

Questa funzionalità è stata introdotta nella versione 9.1 (https://www.postgresql.org/docs/9.1/static/transaction-iso.html)

Implementata con lock di predicato verificato all'atto del commit se più transazioni hanno scritto

Repeatable read vs serializable

Start Transaction isolation level …	
select count(*) from persone	
	Start Transaction isolation level …select count(*) from persone
Insert into persone …	
	Insert into persone …
Commit	Commit

Esercizi proposti

- Verificare il comportamento dei DBMS (almeno uno) rispetto ai livelli di isolamento e alle anomalie
- Ad esempio, verificare che cosa succede, con diversi livelli di isolamento, per gli esempio seguenti (vedere anche presentazione più dettagliata)

Livelli di isolamento
Progetto

- Studiare e sperimentare il comportamento di uno o più DBMS (si consiglia Postgres) nella gestione dei livelli di isolamento, con riferimento alle principali anomalie. Si suggerisce di fare riferimento anche ad esempi simili a quelli utilizzati nei lucidi (considerarli tutti, perché potrebbero essere trattati nelle prove d'esame o in quelle intermedie).
- Dal punto di vista realizzativo, si può procedere sviluppando applicazioni in Java con JDBC, oppure attraverso stored procedure oppure anche utilizzando gli ambienti interattivi. Consegnare una breve relazione che illustri i test fatti, mostrando i risultati dei test stessi (file di output o schermate o anche descrizione delle schermate).